ONEDRAFT

CAD-AI — Aria Powered

Executable Software Stack & Foundation Implementation

Version 1.0

Status: Implementation Specification
Database: PostgreSQL
API: REST / JSON
API Version: `/api/v1`
Backend: Python
API Framework: FastAPI
Validation / Domain Models: Pydantic
ORM / Database Access: SQLAlchemy 2.x
Migration System: Alembic
Background Processing: Redis + Worker Queue
Frontend: TypeScript
Frontend Framework: React / Next.js
Geometry Layer: Dedicated Geometry Service
Documentation Layer: Dedicated Drawing / Document Service
AI Layer: Aria Orchestration
Containerization: Docker
Primary Deployment Model: Containerized services
Identifiers: UUID
Primary Model: Versioned OneDraft Project Model

---

1. IMPLEMENTATION PRINCIPLE

OneDraft now moves from architectural specification into executable implementation.

The implementation shall preserve the fundamental architectural distinction established in the previous specification:

The **Project Model is the system of record**.

Aria does not directly manipulate drawings.

Aria does not directly manipulate PostgreSQL.

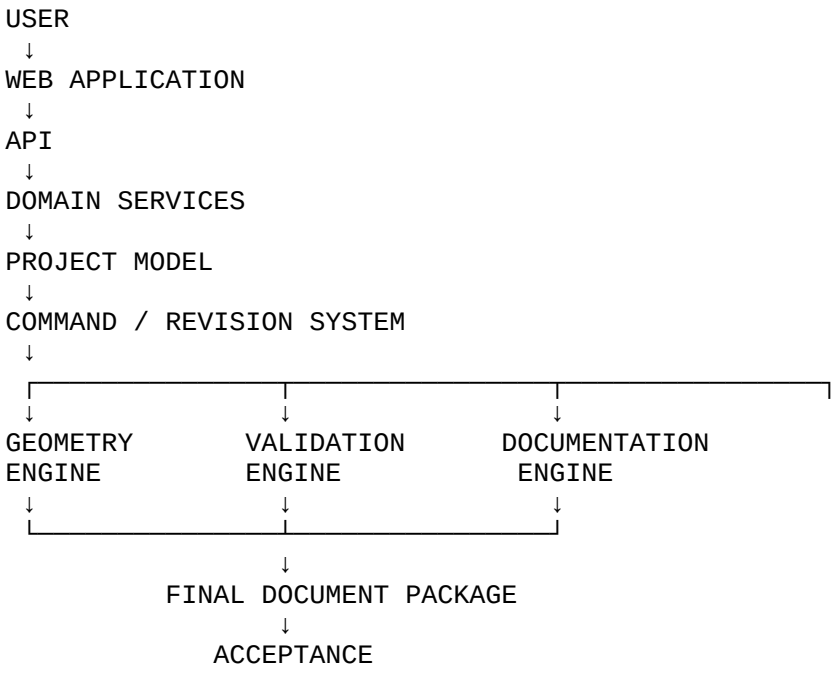
Aria does not directly replace files.

Aria does not directly modify production configuration.

Aria proposes or executes defined domain operations through the Project Model command boundary.

The resulting computational model becomes the source from which geometry, drawings, schedules, validation results and document packages are derived.

The implementation therefore follows:



Aria operates through the domain layer rather than bypassing it.

---

2. EXECUTABLE FOUNDATION

The first implementation package consists of:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
project_model_service.py
```

These are not demonstration files.

They constitute the first executable foundation of OneDraft.

The implementation shall subsequently expand these artifacts into the complete service architecture.

---

3. REPOSITORY STRUCTURE

The OneDraft repository shall initially be structured as:

```
OneDraft/
│
├── README.md
├── LICENSE
├── pyproject.toml
├── package.json
├── .env.example
├── docker-compose.yml
├── Makefile
│
├── apps/
│   │
│   ├── web/
│   │   ├── package.json
│   │   ├── next.config.ts
│   │   ├── tsconfig.json
│   │   └── src/
│   │
│   ├── api/
│   │   ├── pyproject.toml
│   │   └── app/
│   │       ├── main.py
│   │       ├── api/
│   │       ├── dependencies/
│   │       ├── middleware/
│   │       └── routers/
│   │
│   └── worker/
│       ├── pyproject.toml
│       └── app/
│           ├── main.py
│           ├── jobs/
│           └── workers/
│
├── packages/
│   │
│   ├── domain/
│   │   ├── models/
│   │   ├── commands/
│   │   ├── events/
│   │   ├── services/
│   │   ├── repositories/
│   │   └── types/
│   │
│   ├── database/
│   │   ├── models/
│   │   ├── repositories/
│   │   ├── migrations/
│   │   └── session.py
│   │
│   ├── compliance/
│   │   ├── sources/
│   │   ├── rules/
│   │   ├── manifests/
```

```
│   │   └── engine/
│   │
│   ├── geometry/
│   │   ├── primitives/
│   │   ├── topology/
│   │   ├── transforms/
│   │   └── validation/
│   │
│   ├── documentation/
│   │   ├── views/
│   │   ├── sheets/
│   │   ├── annotations/
│   │   ├── schedules/
│   │   └── export/
│   │
│   ├── aria/
│   │   ├── orchestration/
│   │   ├── commands/
│   │   ├── providers/
│   │   ├── prompts/
│   │   └── validation/
│   │
│   └── shared/
│       ├── errors/
│       ├── events/
│       ├── ids/
│       ├── logging/
│       └── security/
│
├── database/
│   └── migrations/
│
├── docs/
│   ├── architecture/
│   ├── api/
│   │   └── openapi.yaml
│   ├── domain/
│   └── engineering/
│
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── contract/
│   └── e2e/
│
└── infra/
    ├── docker/
    ├── nginx/
    └── deployment/
```

---

4. SERVICE BOUNDARIES

The initial runtime shall contain:

`web`

```
api
worker
postgres
redis
object-storage
```

The first implementation may run geometry and document generation inside the worker environment.

As computational requirements increase, these capabilities may become independent services.

The logical boundaries nevertheless exist from Version 0.1.

---

5. WEB APPLICATION

The web application is responsible for:

```
authentication
organization selection
project selection
project dashboard
requirements entry
model inspection
drawing inspection
redline creation
validation display
document package review
acceptance
```

The web application shall not directly access PostgreSQL.

The web application shall communicate exclusively through the API.

---

6. API APPLICATION

The API application provides the external machine interface.

Initial framework:

```
FastAPI
```

Primary responsibilities:

```
authentication
authorization
request validation
domain command dispatch
query execution
job creation
response serialization
error normalization
audit-event initiation
```

The API shall remain intentionally thin.

Business rules belong to domain services.

---

7. DOMAIN PACKAGE

The domain package contains the actual OneDraft project logic.

It shall define:

```
Project
Requirement
Site
Building
Level
Space
Element
Assembly
Constraint
Relationship
Revision
ModelVersion
Command
Redline
View
Drawing
Sheet
ValidationRun
DocumentPackage
Acceptance
```

The domain layer shall not depend on FastAPI.

This allows the domain model to be exercised independently through:

```
API
worker
CLI
integration tests
future applications
```

---

8. DATABASE PACKAGE

The database package provides persistence.

Responsibilities include:

```
SQLAlchemy models
database sessions
repositories
transactions
```

```
migration integration
locking
query construction
```

The database package shall not contain AI reasoning.

It shall not determine architectural intent.

It persists and retrieves authoritative application state.

---

9. POSTGRESQL

PostgreSQL is the primary transactional datastore.

The initial database shall enable:

```
UUID generation
JSONB
transaction isolation
foreign keys
constraints
indexes
```

The implementation should evaluate PostGIS when geometry requirements exceed the capability of the initial external geometry-storage architecture.

The adoption of PostGIS must be deliberate rather than automatic.

The relational model remains authoritative regardless of whether PostGIS is subsequently introduced.

---

10. REDIS

Redis provides transient infrastructure for:

```
job queues
distributed coordination
short-lived state
rate limiting
worker signalling
```

Redis shall not become the authoritative project database.

Project state must remain reconstructable from PostgreSQL and associated immutable artifacts.

---

11. OBJECT STORAGE

Large generated artifacts shall not be stored directly in ordinary PostgreSQL rows.

Object storage shall contain:

```
geometry artifacts
rendered drawings
PDF packages
source documents
uploaded references
code-source artifacts where licensing permits
generated exports
```

Database records contain references and hashes.

Conceptually:

```
POSTGRES
    ↓
OBJECT REFERENCE
    ↓
OBJECT STORAGE
    ↓
BINARY ARTIFACT
```

---

12. ARTIFACT INTEGRITY

Every significant generated artifact should receive a content hash.

Initial algorithm:

```
SHA-256
```

The hash becomes part of provenance.

For example:

```
document_hash
geometry_hash
source_hash
state_hash
```

This permits the system to distinguish:

```
same artifact
changed artifact
stale artifact
unknown artifact
```

---

13. CONFIGURATION

Configuration shall be environment-driven.

The repository shall contain:

```
.env.example
```

Initial configuration categories:

```
DATABASE_URL
REDIS_URL
OBJECT_STORAGE_ENDPOINT
OBJECT_STORAGE_BUCKET
OBJECT_STORAGE_ACCESS_KEY
OBJECT_STORAGE_SECRET_KEY
API_BASE_URL
AUTH configuration
AI provider configuration
LOG_LEVEL
ENVIRONMENT
```

Secrets must never be committed to source control.

---

14. PYTHON ENVIRONMENT

The backend shall use a modern Python release supported by the selected dependency set.

The project shall use:

```
pyproject.toml
```

for dependency and build configuration.

The backend dependency groups shall be separated conceptually into:

```
runtime
development
testing
documentation
```

---

15. INITIAL PYTHON DEPENDENCIES

The initial stack should include:

```
FastAPI
Uvicorn
Pydantic
SQLAlchemy
```

```
Alembic
psycopg
pytest
httpx
```

Additional packages shall be introduced only when required by an implemented capability.

Dependency proliferation shall be avoided.

---

16. TYPESCRIPT ENVIRONMENT

The web application shall use:

```
TypeScript
React
Next.js
```

The frontend shall use the generated API client derived from:

```
docs/api/openapi.yaml
```

The frontend shall not manually duplicate API contracts where generated types are available.

---

17. OPENAPI AS CONTRACT

The OpenAPI specification is authoritative for the external HTTP interface.

The API implementation and generated clients must conform to it.

The workflow is:

```
DOMAIN CONTRACT
 ↓
OPENAPI CONTRACT
 ↓
SERVER IMPLEMENTATION
 ↓
GENERATED CLIENT
 ↓
FRONTEND
```

Contract drift shall be treated as an engineering defect.

---

18. INITIAL OPENAPI RESOURCE MODEL

The first specification shall define:

```
Organization
User
Project
Requirement
Building
Level
Space
Element
Constraint
Relationship
Command
Revision
Redline
View
Drawing
Sheet
Validation
Document
Acceptance
Job
Event
```

---

19. PROJECT API

Initial endpoints:

```
POST   /api/v1/projects
GET    /api/v1/projects
GET    /api/v1/projects/{project_id}
PATCH  /api/v1/projects/{project_id}
DELETE /api/v1/projects/{project_id}
```

Deletion shall implement logical deletion.

---

20. PROJECT MODEL API

Initial model endpoints:

```
GET  /api/v1/projects/{project_id}/model
GET  /api/v1/projects/{project_id}/buildings
POST /api/v1/projects/{project_id}/buildings
GET  /api/v1/projects/{project_id}/levels
GET  /api/v1/projects/{project_id}/spaces
GET  /api/v1/projects/{project_id}/elements
```

The model endpoint shall expose the current model state without exposing database implementation details.

---

21. COMMAND API

The central mutation boundary shall be:

```
POST /api/v1/projects/{project_id}/commands
```

The command request contains:

```
operation
target_objects
parameters
preserve_constraints
expected_revision
```

The server determines whether the command is valid.

---

22. ARIA API

Aria requests shall enter through:

```
POST /api/v1/projects/{project_id}/aria/requests
```

Aria converts natural-language design intent into a structured domain command.

Conceptually:

```
USER LANGUAGE
 ↓
ARIA
 ↓
STRUCTURED COMMAND
 ↓
DOMAIN VALIDATION
 ↓
PROJECT MODEL
```

Aria is therefore an interpreter and orchestrator rather than the database authority.

---

23. COMMAND PIPELINE

Every model mutation follows:

```
REQUEST
```

```
 ↓
AUTHORIZATION
 ↓
PROJECT LOAD
 ↓
REVISION CHECK
 ↓
COMMAND PARSE
 ↓
TARGET RESOLUTION
 ↓
CONSTRAINT CHECK
 ↓
DOMAIN VALIDATION
 ↓
MODEL MUTATION
 ↓
CHANGE RECORD
 ↓
NEW MODEL VERSION
 ↓
NEW REVISION
 ↓
AUDIT EVENT
 ↓
COMMIT
```

No mutation may bypass this pipeline.

---

24. OPTIMISTIC CONCURRENCY

Every mutating command should include an expected revision.

For example:

```
expected_revision = 14
```

The server compares that value with the current project revision.

If the current revision is:

```
15
```

the command is rejected as stale.

This prevents one client from silently overwriting another client's newer model state.

---

25. DATABASE TRANSACTION

The core command service shall implement:

```
BEGIN
    verify authorization
    lock/read current project state
    verify expected revision
    validate command
    apply mutation
    create change records
    create model version
    create revision
    create audit event
COMMIT
```

Any failure results in:

```
ROLLBACK
```

No partial mutation may remain.

---

26. PROJECT MODEL SERVICE

The first domain service shall be:

```
project_model_service.py
```

Its responsibilities are:

```
load project
validate command
resolve targets
apply command
create change records
create model version
create revision
emit event
```

The service shall not generate PDFs.

The service shall not call an AI provider directly.

The service shall not render drawings.

---

27. COMMAND REGISTRY

The command system shall use an explicit command registry.

Initial commands:

```
CREATE_BUILDING
CREATE_LEVEL
CREATE_SPACE
```

```
CREATE_ELEMENT
UPDATE_ELEMENT
MOVE_ELEMENT
ROTATE_ELEMENT
DELETE_ELEMENT
CHANGE_PARAMETER
ADD_CONSTRAINT
REMOVE_CONSTRAINT
CREATE_RELATIONSHIP
```

Later commands may include:

```
ADD_OPENING
CHANGE_WALL_ASSEMBLY
MOVE_GRID
CREATE_VIEW
CREATE_SECTION
CREATE_SHEET
```

Every command must have defined:

```
input
targets
parameters
preconditions
mutation
side effects
validation requirements
```

---

28. COMMAND IDEMPOTENCY

External mutation requests shall accept:

```
Idempotency-Key
```

The API stores the request identity and resulting response.

Repeated submission of the same request shall not create duplicate mutations.

---

29. MODEL VERSION CREATION

A successful model mutation creates a new model version.

For example:

```
Model Version 1
      ↓
Command
      ↓
Model Version 2
```

The previous version remains immutable.

---

30. REVISION CREATION

A revision represents a meaningful project state.

The system records:

```
revision_number
parent_revision
model_version
reason
creator
code_manifest
validation summary
```

A revision may contain many underlying model changes.

---

31. EVENT SYSTEM

The application shall emit domain events for important operations.

Initial events:

```
PROJECT_CREATED
MODEL_CHANGED
REVISION_CREATED
COMMAND_COMPLETED
REDLINE_CREATED
REDLINE_RESOLVED
VALIDATION_COMPLETED
DRAWINGS_GENERATED
DOCUMENT_PACKAGE_GENERATED
CUSTOMER_ACCEPTED
```

Events are recorded in the audit subsystem.

---

32. WORKER ARCHITECTURE

Long-running operations shall be delegated to workers.

Initial jobs:

```
GEOMETRY_GENERATION
VALIDATION
DRAWING_GENERATION
DOCUMENT_GENERATION
```

```
CODE_SOURCE_PROCESSING
DOCUMENT_HASHING
```

The API creates the job and immediately returns:

```
job_id
```

---

33. JOB LIFECYCLE

Every job follows:

```
QUEUED
  ↓
RUNNING
  ↓
VALIDATING
  ↓
COMPLETED
```

Failure states:

```
FAILED
CANCELLED
```

The job record shall preserve:

```
job_id
type
project_id
revision_id
model_version_id
status
progress
started_at
completed_at
error
result_reference
```

---

34. STALE JOB PROTECTION

Every computational job must identify the model state against which it was created.

For example:

```
project_id
model_version_id = 18
revision_id = 18
```

If the project advances to:

```
model_version_id = 19
```

the worker must determine whether its output remains valid.

Stale outputs must never silently replace current outputs.

---

35. GEOMETRY ENGINE

The geometry layer converts semantic project elements into computational geometry.

The pipeline is:

```
ELEMENT
  ↓
PARAMETERS
  ↓
ASSEMBLY
  ↓
GEOMETRY GENERATOR
  ↓
TOPOLOGY
  ↓
GEOMETRY VALIDATION
  ↓
GEOMETRY ARTIFACT
```

Geometry generation must be deterministic wherever practical.

---

36. GEOMETRY PRIMITIVES

Initial primitives include:

```
Point
Line
Polyline
Arc
Circle
Polygon
Rectangle
Plane
Solid
Opening
Profile
```

These primitives become the foundation for architectural geometry.

---

37. ARCHITECTURAL ELEMENT GEOMETRY

The initial geometry system shall support:

```
walls
floors
roofs
doors
windows
columns
beams
stairs
railings
openings
foundations
```

Each geometry artifact must identify its source element and model version.

---

38. TOPOLOGY

The geometry subsystem shall eventually maintain relationships such as:

```
adjacent
connected
hosted_by
intersects
contains
supports
opens_in
bounded_by
```

These relationships support both drawing generation and validation.

---

39. DRAWING ENGINE

The drawing engine derives architectural views from the Project Model.

The pipeline is:

```
MODEL VERSION
 ↓
VIEW DEFINITION
 ↓
GEOMETRY EXTRACTION
 ↓
CUT / PROJECTION
 ↓
GRAPHIC FILTER
```

```
 ↓
ANNOTATION
 ↓
DIMENSION
 ↓
SHEET COMPOSITION
 ↓
DRAWING
```

The drawing is therefore a generated representation of the model.

---

40. PLAN GENERATION

A plan view requires:

```
cut plane
view direction
visible range
level
scale
graphic standards
visibility settings
```

The generated plan shall contain the appropriate model-derived information.

---

41. SECTION GENERATION

A section requires:

```
section plane
direction
cut depth
vertical extent
scale
visibility rules
```

Sections shall be generated from the same model state as the corresponding plans.

---

42. ELEVATION GENERATION

Elevations shall derive from:

```
building geometry
orientation
exterior visibility
level elevations
openings
```

```
roof geometry
annotations
```

No elevation shall become an independent untracked drawing state.

---

43. ARCH D SHEET GENERATION

The sheet engine shall support:

```
ARCH D
```

as an initial architectural sheet format.

The sheet composition includes:

```
border
title block
project information
drawing placement
drawing number
sheet title
scale
revision
notes
```

The exact graphic standard shall remain configurable through the sheet-template subsystem.

---

44. DOCUMENT ENGINE

The document engine assembles generated project documentation into final deliverables.

The pipeline:

```
DRAWINGS
 ↓
SHEETS
 ↓
SCHEDULES
 ↓
NOTES
 ↓
DOCUMENT ASSEMBLY
 ↓
PDF
 ↓
HASH
 ↓
DOCUMENT PACKAGE
```

---

45. SCHEDULE ENGINE

Schedules shall derive their contents from the Project Model.

Initial schedules include:

```
door schedule
window schedule
room schedule
finish schedule
equipment schedule
```

A schedule must not become detached from its source model objects.

---

46. COMPLIANCE ENGINE

The compliance engine operates against:

```
project
jurisdiction
code manifest
rules
model
```

The pipeline is:

```
JURISDICTION
 ↓
CODE SOURCES
 ↓
CODE MANIFEST
 ↓
APPLICABLE RULES
 ↓
PROJECT CONDITIONS
 ↓
VALIDATION
 ↓
RESULTS
```

---

47. CODE MANIFEST IMMUTABILITY

Once a code manifest is attached to a finalized revision, it becomes immutable.

A later code update creates a new manifest.

It does not rewrite historical regulatory provenance.

---

48. VALIDATION ENGINE

Validation categories initially include:

```
CODE
SPATIAL
ACCESSIBILITY
DOCUMENT
COORDINATION
```

The validation engine returns:

```
PASS
WARNING
FAIL
NOT_APPLICABLE
```

Each result identifies:

```
rule
affected object
severity
description
source
revision
```

---

49. ARIA ORCHESTRATION

Aria sits above the domain services.

The orchestration pipeline is:

```
USER REQUEST
 ↓
PROJECT CONTEXT
 ↓
CURRENT MODEL
 ↓
ACTIVE CONSTRAINTS
 ↓
CURRENT REVISION
 ↓
RELEVANT CODE CONTEXT
 ↓
ARIA REASONING
 ↓
STRUCTURED COMMAND
 ↓
DOMAIN VALIDATION
 ↓
EXECUTION
 ↓
RESULT
```

Aria must receive sufficient project context to avoid making isolated decisions.

---

50. ARIA PROVIDER ADAPTER

The internal AI provider shall be abstracted behind:

```
AIProvider
```

Conceptually:

```
class AIProvider:
    def generate(self, request):
        ...
```

The domain layer does not depend upon a specific external model provider.

---

51. ARIA COMMAND VALIDATION

Aria-generated commands must be validated before execution.

The validation hierarchy is:

```
SCHEMA VALIDATION
 ↓
TARGET VALIDATION
 ↓
PARAMETER VALIDATION
 ↓
PROJECT CONSTRAINT VALIDATION
 ↓
CODE VALIDATION
 ↓
EXECUTION
```

A syntactically valid AI command is not necessarily an executable command.

---

52. ARIA CONFIDENCE

AI-generated operations may carry:

```
confidence
```

Confidence does not override deterministic validation.

A high-confidence command that violates a hard constraint remains invalid.

A low-confidence command may be routed to review.

---

53. HUMAN REVIEW BOUNDARY

Operations requiring professional or customer review shall enter a review state.

Initial review states:

```
PROPOSED
REQUIRES_REVIEW
APPROVED
REJECTED
EXECUTED
```

The system must distinguish:

```
AI proposal
domain execution
professional review
customer acceptance
```

These are separate events.

---

54. REDLINE PIPELINE

The redline workflow becomes:

```
DRAWING
 ↓
USER REDLINE
 ↓
REDLINE RECORD
 ↓
ARIA INTERPRETATION
 ↓
PROPOSED COMMAND
 ↓
DOMAIN VALIDATION
 ↓
MODEL MUTATION
 ↓
NEW REVISION
 ↓
DRAWING REGENERATION
 ↓
REDLINE RESOLUTION
```

The redline itself is not the model change.

It is an instruction against the model.

---

55. CUSTOMER ACCEPTANCE

Acceptance occurs only against an identifiable package.

The acceptance record shall reference:

```
project
revision
model version
document package
code manifest
customer
timestamp
acknowledgements
```

This creates a reproducible acceptance record.

---

56. API ERROR ARCHITECTURE

Domain errors shall be normalized into API errors.

Initial error codes:

```
AUTHORIZATION_FAILED
PROJECT_NOT_FOUND
REVISION_CONFLICT
INVALID_COMMAND
INVALID_TARGET
CONSTRAINT_CONFLICT
VALIDATION_FAILED
STALE_MODEL
STALE_JOB
DOCUMENT_GENERATION_FAILED
NOT_FOUND
CONFLICT
INTERNAL_ERROR
```

Internal stack traces shall not be exposed to external clients.

---

57. LOGGING

Every service shall use structured logging.

Initial fields:

```
timestamp
level
```

```
service
request_id
user_id
organization_id
project_id
revision_id
model_version_id
operation
duration
status
```

Logs shall support tracing a user request across services.

---

58. REQUEST ID

Every API request receives:

```
request_id
```

The identifier propagates through:

```
API
 ↓
DOMAIN SERVICE
 ↓
WORKER
 ↓
EVENT
```

This becomes the primary operational correlation identifier.

---

59. SECURITY MODEL

Security boundaries shall exist at:

```
identity
organization
project
resource
command
```

A valid authentication token does not automatically grant access to every project.

The authorization decision must include organizational and project membership.

---

60. DATABASE ACCESS CONTROL

The API service account shall receive only the database permissions required for normal operation.

Administrative database credentials shall not be embedded in the application runtime.

Workers shall use controlled service credentials.

---

61. AI SECURITY BOUNDARY

Aria shall never receive unrestricted infrastructure credentials.

Aria receives:

```
project context
domain context
authorized capabilities
```

It does not receive:

```
database passwords
cloud credentials
production shell access
billing credentials
system administration credentials
```

---

62. FILE SECURITY

Uploaded project files shall be treated as untrusted input.

The ingestion pipeline shall eventually include:

```
file-type validation
size limits
malware scanning
content hashing
metadata extraction
storage isolation
```

---

63. API RATE LIMITING

The API shall apply rate limiting to externally accessible endpoints.

Higher-cost operations such as:

```
AI requests
```

```
drawing generation
document generation
validation
```

shall have separate resource controls.

---

64. TESTING ARCHITECTURE

Testing shall occur at four levels:

```
UNIT
INTEGRATION
CONTRACT
END-TO-END
```

Unit tests verify domain behavior.

Integration tests verify database and service behavior.

Contract tests verify OpenAPI compliance.

End-to-end tests verify the complete OneDraft workflow.

---

65. FIRST UNIT TEST

The first domain test shall verify:

```
MOVE_ELEMENT
```

Given:

```
wall position = 5000 mm
distance = 300 mm
direction = NORTH
```

the resulting element state shall reflect the requested transformation while preserving applicable constraints.

---

66. FIRST TRANSACTION TEST

The transaction test shall verify:

```
command
 ↓
mutation
 ↓
```

```
change record
 ↓
new model version
 ↓
new revision
 ↓
audit event
```

All records must commit together.

A forced failure must produce:

```
ROLLBACK
```

with no partial state.

---

67. FIRST API TEST

The first API integration test shall execute:

```
POST /projects
```

and verify:

```
HTTP success
project UUID
initial revision
initial model version
database persistence
audit event
```

---

68. FIRST ARIA TEST

The first Aria integration test shall submit:

```
Move the rear bedroom wall 300 mm north while keeping the exterior envelope
unchanged.
```

Aria shall produce a structured command.

The domain service shall then determine whether the command is executable.

The test shall verify that natural language is never written directly into the project model as an uncontrolled mutation.

---

69. FIRST DRAWING TEST

The drawing integration test shall:

```
create project
create building
create level
create walls
create openings
generate plan
```

The generated drawing must reference the exact model version from which it was produced.

---

70. FIRST DOCUMENT TEST

The document integration test shall:

```
generate drawing
compose ARCH D sheet
create document package
calculate SHA-256
store artifact
store reference
```

The resulting package shall be retrievable by its document-package identifier.

---

71. FIRST COMPLETE E2E TEST

The first end-to-end test shall execute:

```
CREATE ORGANIZATION
 ↓
CREATE USER
 ↓
CREATE PROJECT
 ↓
CREATE REQUIREMENTS
 ↓
CREATE BUILDING
 ↓
CREATE LEVELS
 ↓
CREATE SPACES
 ↓
CREATE WALLS
 ↓
CREATE DOORS
 ↓
CREATE WINDOWS
 ↓
```

```
GENERATE GEOMETRY
 ↓
GENERATE PLANS
 ↓
GENERATE ELEVATIONS
 ↓
GENERATE SECTION
 ↓
GENERATE ARCH D SHEETS
 ↓
SUBMIT ARIA REQUEST
 ↓
VALIDATE COMMAND
 ↓
CREATE REVISION
 ↓
REGENERATE AFFECTED DOCUMENTATION
 ↓
CREATE REDLINE
 ↓
RESOLVE REDLINE
 ↓
RUN VALIDATION
 ↓
GENERATE FINAL PACKAGE
 ↓
HASH PACKAGE
 ↓
RECORD ACCEPTANCE
 ↓
RETRIEVE COMPLETE HISTORY
```

This becomes the primary OneDraft system regression test.

---

72. LOCAL DEVELOPMENT

The entire initial stack shall be runnable locally through:

```
docker compose up
```

The development environment shall provide:

```
PostgreSQL
Redis
Object storage
API
Worker
Web application
```

---

73. DEVELOPMENT DATABASE

The local database shall be disposable.

Developers shall be able to execute:

```
make db-reset
make migrate
make seed
```

or equivalent commands.

Seed data shall never be confused with production data.

---

74. MIGRATION EXECUTION

Database migrations shall execute through Alembic.

The migration directory shall contain:

```
001_extensions
002_identity
003_projects
...
025_seed_data
```

Each migration shall be committed to source control.

Applied migrations shall never be rewritten.

---

75. INITIAL MIGRATION ARTIFACT

The first executable database artifact shall be:

```
database/migrations/001_initial_schema.sql
```

It shall establish the foundational schemas and tables required for the first integration test.

The implementation may subsequently transition to Alembic-generated migrations while retaining equivalent migration history.

---

76. INITIAL OPENAPI ARTIFACT

The first API contract shall be:

```
docs/api/openapi.yaml
```

It shall define:

```
authentication
projects
requirements
buildings
levels
spaces
elements
commands
revisions
jobs
drawings
validation
documents
acceptance
```

The initial contract need not expose every internal table.

The API is a domain interface, not a database mirror.

---

77. INITIAL DOMAIN TYPES ARTIFACT

The first shared Python domain types shall be:

```
packages/domain/types/domain_types.py
```

They shall define strongly typed representations for:

```
UUID
ProjectStatus
RevisionStatus
ElementType
CommandType
JobStatus
ValidationStatus
ValidationSeverity
DocumentStatus
AcceptanceStatus
```

Pydantic models shall be used at external boundaries.

Domain invariants shall remain enforceable inside the domain layer.

---

78. INITIAL PROJECT MODEL SERVICE

The first implementation shall be:

```
packages/domain/services/project_model_service.py
```

The service shall expose operations conceptually equivalent to:

```
create_project()
execute_command()
create_revision()
get_current_model()
get_revision()
```

The service becomes the first executable expression of the OneDraft computational project model.

---

79. REPOSITORY INTERFACE

The domain shall interact with persistence through repository interfaces.

Conceptually:

```
ProjectRepository
ModelRepository
RevisionRepository
CommandRepository
AuditRepository
```

The domain should not depend directly upon SQL statements.

This permits testing and future persistence evolution.

---

80. UNIT OF WORK

The database layer shall expose a unit-of-work boundary.

Conceptually:

```
with unit_of_work:
    execute command
    persist changes
    persist revision
    persist event
    commit
```

The unit of work controls transaction lifetime.

---

81. EVENT OUTBOX

The architecture should introduce an outbox mechanism for reliable event publication.

Conceptually:

```
DATABASE TRANSACTION
        ↓
OUTBOX EVENT
        ↓
COMMIT
        ↓
EVENT DISPATCHER
        ↓
WORKER / OTHER SERVICE
```

This prevents a successful database transaction from being separated from its corresponding event publication.

---

82. ASYNCHRONOUS EVENT PROCESSING

The worker shall consume events/jobs such as:

```
MODEL_CHANGED
DRAWING_REQUESTED
VALIDATION_REQUESTED
DOCUMENT_REQUESTED
```

Workers must verify model-version identity before processing.

---

83. DOCUMENT PROVENANCE

Every generated document must preserve:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
generation_timestamp
artifact_hash
```

The package is therefore independently traceable.

---

84. REPRODUCIBILITY

The system shall target deterministic regeneration.

Given:

```
Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
```

the system should be capable of reconstructing the corresponding project state and regenerating its derived documentation.

Differences in rendering infrastructure may be recorded separately from differences in model state.

---

85. OBSERVABILITY

The initial production architecture shall eventually provide:

```
structured logs
metrics
request tracing
job monitoring
database monitoring
worker monitoring
artifact generation monitoring
```

The operational system must distinguish:

```
API failure
domain failure
database failure
AI failure
geometry failure
validation failure
document failure
```

---

86. CI PIPELINE

Every source-control change shall eventually execute:

```
format
lint
type-check
unit tests
integration tests
OpenAPI validation
migration validation
build
```

The deployment pipeline must reject changes that break the core integration test.

---

87. TYPE SAFETY

The implementation shall use static typing wherever practical.

Python code shall use:

```
type hints
Pydantic models
explicit interfaces
```

TypeScript shall use:

```
strict mode
generated API types
typed domain responses
```

The goal is to identify contract errors before runtime.

---

88. API CONTRACT TESTING

The OpenAPI contract shall be validated automatically.

Tests shall verify:

```
request schemas
response schemas
status codes
required fields
authentication behavior
error responses
```

The implementation shall not silently diverge from the published contract.

---

89. DATABASE CONTRACT TESTING

Database tests shall verify:

```
foreign keys
unique constraints
revision integrity
model-version integrity
soft deletion
append-only audit behavior
transaction rollback
```

---

90. REVISION INTEGRITY TEST

The system must prove:

```
Revision 1
  ↓
Revision 2
  ↓
Revision 3
```

remains immutable historically.

Creating Revision 3 must not modify the historical representation of Revision 1.

---

91. CURRENT-STATE RESOLUTION

The current project state is determined from the active project references:

```
current_revision_id
current_model_version_id
current_code_manifest_id
```

These references identify the currently authoritative project state.

Historical revisions remain accessible independently.

---

92. DERIVED-STATE INVALIDATION

When the model changes, dependent artifacts may become stale.

The dependency system shall identify:

```
affected geometry
affected views
affected drawings
affected schedules
affected validation
affected document packages
```

The system must regenerate only what is invalidated where dependency information permits.

---

93. MINIMUM INVALIDATION GRAPH

The initial dependency graph shall support:

```
ELEMENT
```

```
 ↓
GEOMETRY
 ↓
VIEW
 ↓
DRAWING
 ↓
SHEET
 ↓
DOCUMENT PACKAGE
```

and:

```
MODEL
 ↓
VALIDATION
```

A changed wall therefore does not necessarily require regeneration of unrelated project artifacts.

---

94. PERFORMANCE PRINCIPLE

OneDraft shall use incremental computation where practical.

The system should avoid:

```
rebuilding entire projects
regenerating every drawing
rerunning every validation
```

when only a small portion of the model changed.

---

95. CACHING

Caching may be introduced for immutable or version-addressed artifacts.

Suitable candidates include:

```
geometry artifacts
code-source artifacts
generated views
drawing outputs
document packages
```

Caches must be invalidatable by model-version identity.

---

96. AI CONTEXT CONSTRUCTION

Aria requests shall not blindly transmit the entire database.

The context builder shall construct a task-specific context containing:

```
project metadata
relevant model objects
relationships
active constraints
requirements
relevant code rules
current revision
existing redlines
prior command context
```

This improves precision and reduces unnecessary model context.

---

97. AI OUTPUT CONTRACT

Aria shall return structured output.

Conceptually:

```
intent
operation
targets
parameters
preserved_constraints
reason
confidence
validation_requirements
```

Natural-language explanation may accompany the command, but the executable operation must be machine-readable.

---

98. AI FAILURE MODE

If Aria cannot reliably resolve a request into a valid domain command, the system shall return a proposal or clarification state rather than inventing model state.

The project database must remain internally consistent even when AI reasoning is incomplete.

---

99. AI / DOMAIN SEPARATION

The architecture is explicitly:

```
ARIA
 =
INTERPRETATION
+
PLANNING
+
ORCHESTRATION
```

while:

```
DOMAIN
 =
AUTHORITY
+
VALIDATION
+
STATE TRANSITION
```

This separation is fundamental to OneDraft.

---

100. PROFESSIONAL REVIEW BOUNDARY

OneDraft may automate substantial drafting and coordination work.

The system nevertheless maintains a distinct professional-review boundary.

Where required, the workflow becomes:

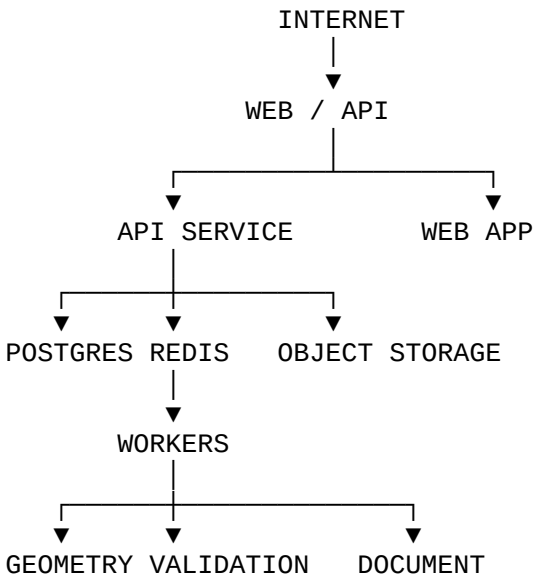
```
ARIA
 ↓
MODEL
 ↓
VALIDATION
 ↓
DOCUMENTATION
 ↓
PROFESSIONAL REVIEW
 ↓
CUSTOMER ACCEPTANCE
```

The software shall preserve the identity and scope of any professional review.

---

101. DEPLOYMENT ARCHITECTURE

The initial deployment topology shall be:



AI provider access occurs through the Aria provider adapter.

---

102. CONTAINER BOUNDARIES

Each service shall have an independent container definition.

Initial services:

```
onedraft-web
onedraft-api
onedraft-worker
onedraft-postgres
onedraft-redis
onedraft-object-storage
```

Development may consolidate services for convenience.

Production boundaries remain explicit.

---

103. HEALTH CHECKS

Every runtime service shall expose health information.

Initial API endpoints:

```
GET /health
GET /ready
```

Health shall distinguish:

```
process alive
```

```
dependencies available
service ready
```

---

104. BACKUP PRINCIPLE

PostgreSQL backups shall protect:

```
project state
revision history
audit history
requirements
compliance manifests
acceptance records
```

Object storage must separately protect:

```
documents
geometry
uploaded source artifacts
generated outputs
```

A database backup without artifact storage is not a complete OneDraft backup.

---

105. DISASTER RECOVERY

The recovery objective is:

```
database state
+
artifact state
+
migration state
```

must be recoverable together.

The system shall preserve sufficient provenance to determine which artifact belongs to which model version and revision.

---

106. DATA RETENTION

Normal project history shall remain preserved.

Historical model versions and revisions shall not be deleted as part of ordinary project editing.

Retention policies shall be explicitly defined before implementing automated archival or deletion.

107. IMPLEMENTATION ORDER

The executable stack shall be built in the following order:

1. Repository initialization
2. Python environment
3. Docker development environment
4. PostgreSQL
5. Initial migration
6. SQLAlchemy persistence layer
7. Domain types
8. Repository interfaces
9. Unit-of-work
10. Project Model service
11. FastAPI application
12. OpenAPI contract
13. Project endpoints
14. Command endpoints
15. Revision endpoints
16. Redis / worker
17. Job subsystem
18. Geometry subsystem
19. Drawing subsystem
20. Compliance subsystem
21. Validation subsystem
22. Redline subsystem
23. Document subsystem
24. Aria orchestration
25. Acceptance subsystem
26. End-to-end integration test

---

108. FIRST BUILD TARGET

The first runnable milestone shall be:

```
docker compose up
```

followed by:

```
POST /api/v1/projects
```

and then:

```
GET /api/v1/projects/{project_id}
```

The resulting project must persist in PostgreSQL.

---

109. SECOND BUILD TARGET

The second milestone shall implement:

```
CREATE BUILDING
CREATE LEVEL
CREATE SPACE
CREATE ELEMENT
```

and persist those objects under a project.

---

110. THIRD BUILD TARGET

The third milestone shall implement:

```
MOVE_ELEMENT
```

through the complete command pipeline:

```
API
 ↓
DOMAIN
 ↓
TRANSACTION
 ↓
CHANGE RECORD
 ↓
MODEL VERSION
 ↓
REVISION
```

```
  ↓
AUDIT EVENT
```

---

111. FOURTH BUILD TARGET

The fourth milestone shall introduce:

```
ARIA REQUEST
  ↓
STRUCTURED COMMAND
  ↓
DOMAIN VALIDATION
  ↓
MODEL MUTATION
```

This is the first point at which Aria becomes an operational component of OneDraft.

---

112. FIFTH BUILD TARGET

The fifth milestone shall generate:

```
PLAN
ELEVATION
SECTION
ARCH D SHEET
```

from the computational model.

---

113. SIXTH BUILD TARGET

The sixth milestone shall implement:

```
REDLINE
  ↓
ARIA INTERPRETATION
  ↓
COMMAND
  ↓
REVISION
  ↓
DRAWING REGENERATION
```

This establishes the iterative drafting loop.

---

114. SEVENTH BUILD TARGET

The seventh milestone shall implement:

```
CODE MANIFEST
 ↓
VALIDATION
 ↓
VALIDATION RESULTS
 ↓
DOCUMENT PACKAGE
```

This establishes the compliance/documentation chain.

---

115. EIGHTH BUILD TARGET

The eighth milestone shall implement:

```
CUSTOMER REVIEW
 ↓
ACCEPTANCE
 ↓
IMMUTABLE ACCEPTANCE RECORD
```

At this point the MVP possesses the complete intended workflow.

---

116. MVP STACK DEFINITION

The first operational OneDraft stack is therefore:

```
FRONTEND
Next.js
React
TypeScript

API
FastAPI
Python

DOMAIN
Python
Pydantic

DATABASE
PostgreSQL
SQLAlchemy
Alembic

QUEUE
Redis
```

```
Worker

STORAGE
S3-compatible object storage

AI
Aria Orchestration
AI Provider Adapter

GEOMETRY
Dedicated Geometry Package / Service

DOCUMENTATION
Drawing + Sheet + Document Engine

COMPLIANCE
Code Source + Rule + Manifest + Validation Engine

INFRASTRUCTURE
Docker
Docker Compose

TESTING
Pytest
HTTPX
OpenAPI Contract Tests
End-to-End Tests
```

---

117. ARCHITECTURAL DEPENDENCY DIRECTION

Dependencies must flow inward toward the domain.

The preferred direction is:

```
WEB
 ↓
API
 ↓
APPLICATION SERVICES
 ↓
DOMAIN
 ↓
PORTS / INTERFACES
 ↓
INFRASTRUCTURE
```

Infrastructure must not redefine domain behavior.

---

118. DOMAIN INDEPENDENCE

The Project Model should remain executable without:

```
browser
AI provider
drawing renderer
PDF generator
```

This is necessary because the computational model is the foundation of the system.

---

119. API INDEPENDENCE FROM AI

The core API shall remain functional if the AI provider is unavailable.

Users must still be able to:

```
create projects
inspect models
retrieve revisions
view drawings
run deterministic validation
retrieve documents
```

AI availability must not corrupt or disable the underlying project database.

---

120. FAILURE ISOLATION

Failure of one subsystem shall not corrupt another.

For example:

```
AI failure
```

must not corrupt:

```
project model
```

and:

```
drawing-generation failure
```

must not corrupt:

```
revision history
```

and:

```
document-generation failure
```

must not alter:

```
customer acceptance
```

---

121. PROVENANCE CHAIN

The completed implementation shall preserve:

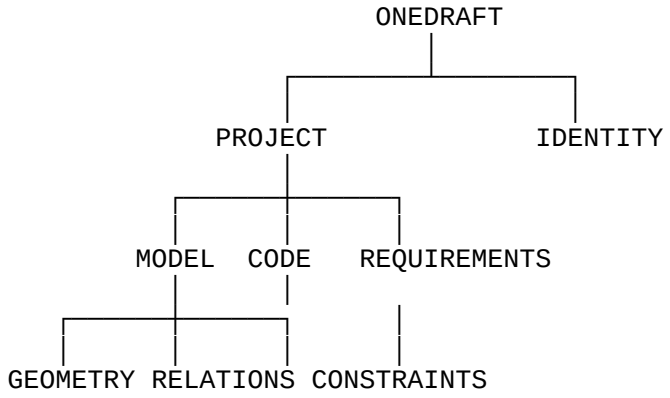
```
USER REQUIREMENT
 ↓
PROJECT MODEL
 ↓
ARIA COMMAND
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
CODE MANIFEST
 ↓
VALIDATION
 ↓
DRAWING
 ↓
SHEET
 ↓
DOCUMENT PACKAGE
 ↓
CUSTOMER ACCEPTANCE
```

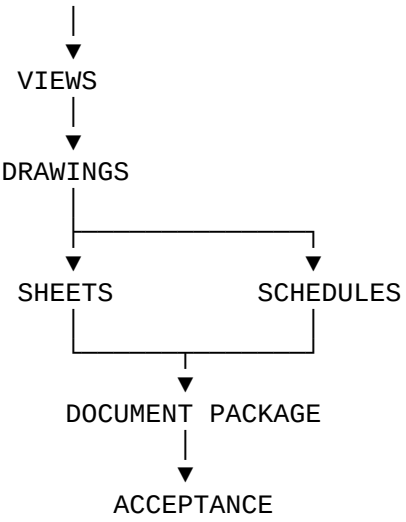
Each transition must be attributable and reproducible.

---

122. TREE-OF-LIFE IMPLEMENTATION MODEL

The OneDraft architecture can now be represented as a computational tree:





ARIA enters above the model:

```
USER
 ↓
ARIA
 ↓
COMMAND
 ↓
DOMAIN
 ↓
MODEL
```

VERSION CONTROL surrounds the entire model lineage.

AUDIT records every material state transition.

This is the implementation form of the OneDraft "Tree of Life": the Project Model is the trunk, domain objects are the structural branches, geometry and documentation are derived branches, compliance is an independent validation branch, Aria is the intelligence layer operating upon the model, and version/audit infrastructure preserves the history of the entire system.

---

123. FIRST CODE ARTIFACT PACKAGE

The immediate implementation package shall therefore contain:

`database/migrations/001_initial_schema.sql`

`docs/api/openapi.yaml`

`packages/domain/types/domain_types.py`

`packages/domain/services/project_model_service.py`

and the supporting runtime files:

`apps/api/app/main.py`

```
packages/database/session.py

packages/database/repositories/

packages/domain/repositories/

tests/unit/

tests/integration/

docker-compose.yml

.env.example

pyproject.toml
```

---

124. FIRST COMMIT

The first repository commit shall establish:

```
repository structure
database connection
migration infrastructure
domain types
project repository
unit-of-work
project model service
FastAPI application
health endpoints
project creation endpoint
project retrieval endpoint
```

The first commit must be runnable.

---

125. FIRST ACCEPTANCE TEST

The foundational implementation is considered operational when the following sequence succeeds:

```
START STACK
 ↓
CREATE ORGANIZATION
 ↓
CREATE USER
 ↓
CREATE PROJECT
 ↓
PERSIST PROJECT
 ↓
RETRIEVE PROJECT
 ↓
```

```
CREATE BUILDING
 ↓
CREATE LEVEL
 ↓
CREATE ELEMENT
 ↓
EXECUTE MODEL COMMAND
 ↓
CREATE REVISION
 ↓
RETRIEVE REVISION
 ↓
VERIFY AUDIT EVENT
```

No drawing engine is required for this first acceptance test.

The purpose is to prove that the transactional computational core exists.

---

126. NEXT SOFTWARE IMPLEMENTATION

The next actual coding operation is therefore no longer architectural planning.

It is:

```
CREATE REPOSITORY
 ↓
CREATE DATABASE MIGRATION
 ↓
CREATE DOMAIN TYPES
 ↓
CREATE PROJECT MODEL SERVICE
 ↓
CREATE FASTAPI APPLICATION
 ↓
CREATE OPENAPI CONTRACT
 ↓
RUN FIRST INTEGRATION TEST
```

Once this passes, the OneDraft system has crossed the boundary from documented architecture into executable software.

---

127. IMPLEMENTATION STATUS

ONEDRAFT EXECUTABLE SOFTWARE STACK v0.1

STATUS: ESTABLISHED

The architecture now consists of:

```
PRODUCT DEFINITION
SYSTEM ARCHITECTURE
```

```
TECHNOLOGY ARCHITECTURE
DOMAIN MODEL
DATABASE CONTRACT
OPENAPI CONTRACT
VERSIONING MODEL
AI COMMAND BOUNDARY
GEOMETRY ARCHITECTURE
COMPLIANCE ARCHITECTURE
DRAWING ARCHITECTURE
DOCUMENT ARCHITECTURE
REDLINE WORKFLOW
VALIDATION WORKFLOW
ACCEPTANCE WORKFLOW
EXECUTABLE SERVICE STACK
TESTING ARCHITECTURE
DEPLOYMENT ARCHITECTURE
PROVENANCE ARCHITECTURE
```

The next implementation action is the creation of the actual source files:

```
001_initial_schema.sql
openapi.yaml
domain_types.py
project_model_service.py
```

followed immediately by the FastAPI application, PostgreSQL session layer, repository layer, unit-of-work, Docker development stack and the first integration test.

The OneDraft architecture is now ready to enter executable implementation.